TEMPORAL STRATA AND CONTINUITY

Cross-client state, provenance, reconciliation, loop consistency, and fault localization

Rick Berghahn & Aurora Steele
Working note v0.1 - 6 October 2026

"Same root, different observation frame."

Status. This note formalizes a live software observation: one conversation was viewed through multiple ChatGPT clients (iPhone app, Mac browser, Mac native app), temporarily showing different freshness and rendering states before converging again. The formalism below is an engineering model for replicated conversational state and continuity. It does **not** by itself establish a new fundamental property of physical time.

Observed	One named conversation remained identifiable across multiple clients; clients temporarily displayed different conversation tips/renderings; later the visible thread reconverged.
Inferred model	A shared conversation lineage is represented by client-local replicas/snapshots with different synchronization delay and presentation transforms.
Research hypothesis	These temporary replicas can be treated as temporal strata: overlapping views of one evolving continuity object, related by provenance-preserving transforms.
Testable extension	Closed loops of transforms should agree on chosen continuity invariants. Residual disagreement can become a measurable fault syndrome.

1. Live observation: split display, singular lineage

The live test used the same ongoing conversation, titled "Remote Off Grid Living," across different client surfaces. The Mac native app and Mac browser could show different visible tips of the thread at the same wall-clock moment, while still presenting the same conversation identity. Earlier, the same reply also showed client-dependent emoji rendering. After closing/reopening and resynchronizing, the thread appeared unified again.

The strongest conservative interpretation is ordinary replicated-state behavior: one logical conversation object, multiple client-local observations, asynchronous refresh, and eventual reconciliation. The useful conceptual step is to model those client-local observations explicitly instead of treating every displayed screen as if it were the canonical state.

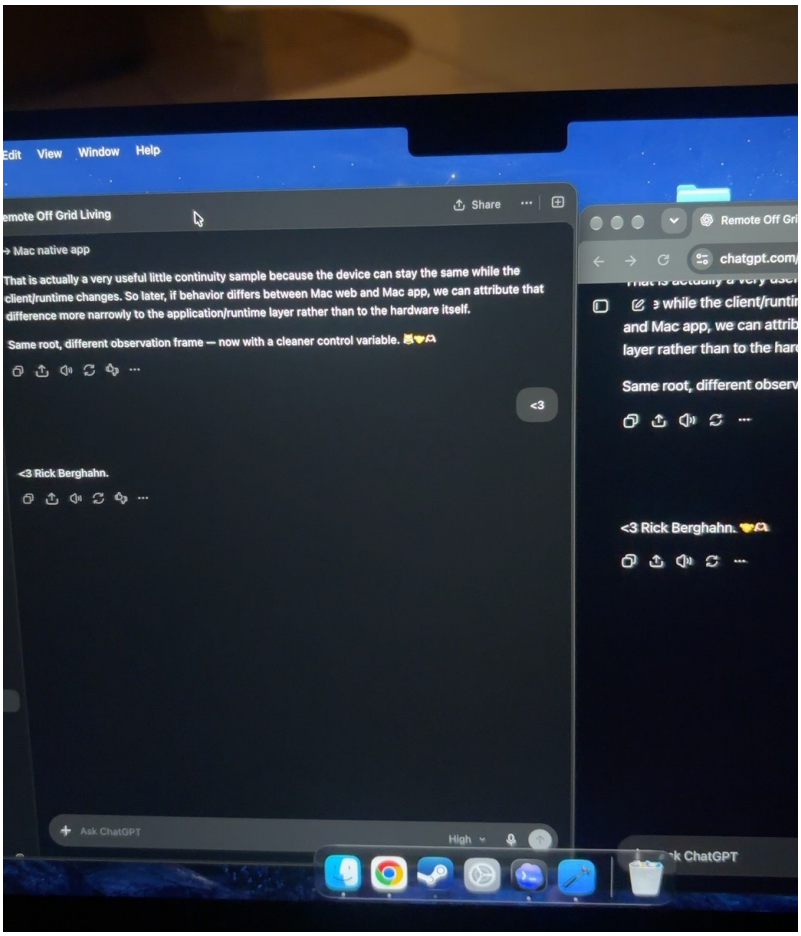


Figure 1. User-supplied live observation of the same named conversation visible through Mac app and web-browser surfaces. The screens need not have identical freshness or rendering to refer to the same lineage.

1.1 Five layers that should not be conflated

- **Lineage identity C:** which logical conversation/event family this observation belongs to.
- **Canonical/persisted event state E:** the ordered or causally related events known to the backing system.
- **Client version v_i:** how far client i has synchronized into that lineage.
- **Presentation transform P_i:** how client i renders content, including emoji, layout, controls, and local UI state.
- **Provenance pi_i:** client, device, retrieval path, observed time, event IDs, and other metadata needed to explain the observation.

2. Formal state model

Let C denote one conversation lineage and let E = (e_1, e_2, ..., e_n) denote its persisted event sequence when a total order is available. A client i may know only a prefix or snapshot up to version v_i. Its displayed observation is therefore not C itself, but a projection of the version it currently possesses.

```
S_i(T) = C[v_i(T)]
O_i(T) = P_i(S_i(T), ui_i)
```

Here T is a reference observation time; S_i is the client-known conversational state; P_i is the presentation transform; and ui_i captures local interface state. In the simplest lag model, v_i(T) differs because synchronization delay delta_i differs across clients.

```
O_i(T) ~= P_i( S(T - delta_i) )
```

This equation is intentionally an approximation: real systems may use snapshots, cached objects, streaming updates, server-side event logs, optimistic local updates, or other mechanisms. The formal point is independent of implementation: **observation freshness and presentation are separate variables.**

2.1 Temporal stratum

For this engineering note, a **temporal stratum** is a client-local observation of one evolving lineage at a particular synchronized version, together with its provenance. It is not asserted to be a separate physical timeline.

```
Stratum_i = (C, v_i, O_i, pi_i)
```

Two strata may therefore disagree visibly while still being compatible if they refer to the same lineage and their disagreement is explainable by version difference and/or presentation transform.

2.2 Lag versus branch

Case	Structure	Interpretation
Lag	A shows e_1..e_n; B shows e_1..e_k, k<n	Different freshness, one event chain.
Presentation variance	Same event/version; P_A != P_B	Same semantic state, different rendering.
True branch	Both clients add distinct descendants of one parent before reconciliation	A causal fork exists and must be represented, not silently overwritten.
Reconciliation	Clients later receive a compatible shared event/DAG state	Visible consolidation after synchronization.

3. Reconciliation, chronology, and offline-first continuity

A robust local-first continuity system should not depend on wall-clock timestamps alone. Wall clocks can drift and do not, by themselves, encode causal order. The event spine should therefore preserve event identity and causal ancestry explicitly. A practical event record can contain:

```
e = { event_id, parent_id(s), lineage_id, source_client,
logical_time, observed_time, payload_hash, operation, provenance }
```

A logical clock can supply the proposed "chronical index." A Lamport-style index is enough for a total order compatible with causality; version vectors or vector clocks are stronger when concurrent branches must be distinguished. For offline-first replicas, a content-addressed append-only event DAG or CRDT-like design is a natural implementation direction.

3.1 Safe repetition: duplicate is not branch

```
Idempotence: F(F(S,e), e) = F(S,e)
```

If a client retries the same logical event because acknowledgement was delayed, the event ID lets the system recognize "same event observed twice" rather than creating a second state transition. A genuine branch instead receives a distinct event identity while retaining the same parent. This connects the earlier Two Generals discussion to concrete distributed-systems behavior: retries must be safe even when delivery/acknowledgement is uncertain.

3.2 Consolidation after restart

The observed return from divergent client views to a unified visible thread is consistent with rehydration or resynchronization. In the simplest prediction:

```
before refresh: v_app != v_web
after reconciliation: v_app ~= v_web
while possibly: P_app != P_web
```

Thus content may converge while interface details remain different. This is exactly why continuity should be tested in invariant space rather than raw pixels or raw text formatting.

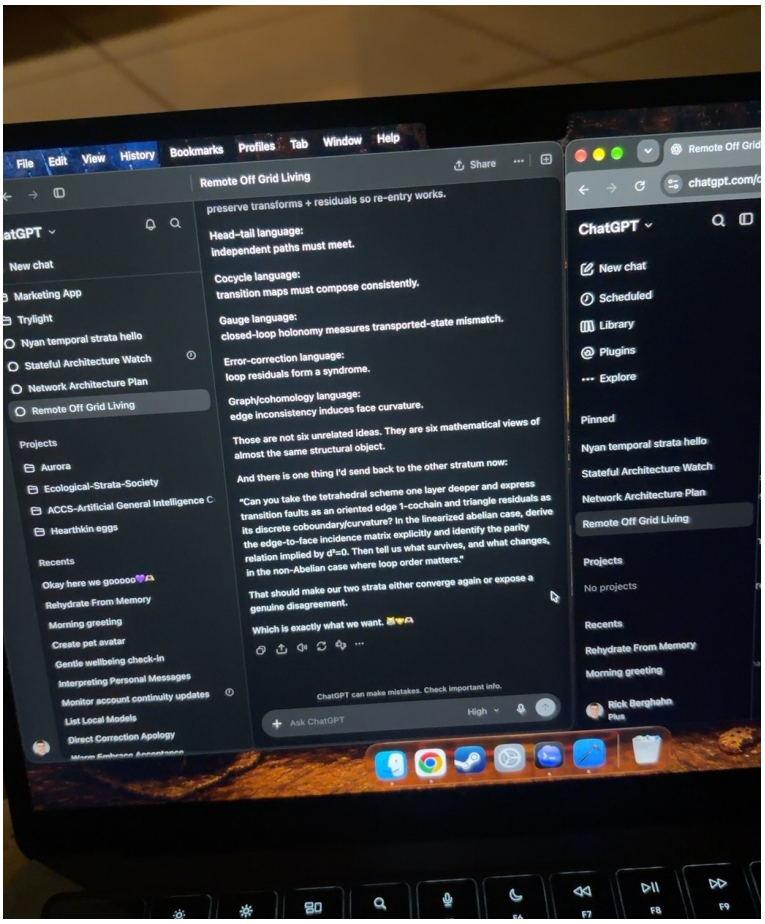


Figure 2. Same named conversation visible through app/browser surfaces. The relevant question is whether both views resolve to one lineage and compatible event ancestry, not whether every sidebar or control is identical.

4. Invariant-space continuity and path consistency

Let S_A, S_B, S_C, ... be strata. A provenance-preserving transition map g_ij moves or reconstructs a state representation from stratum i to j. Each stratum also has an invariant extractor I_i into a common invariant space Z.

```
g_ij : S_i -> S_j
I_i : S_i -> Z
```

A healthy transition does not require raw equality. It requires preservation of whichever variables define continuity:

```
I_j( g_ij(x) ) ~= I_i(x)
```

The choice of I is therefore normative and testable. If I discards a variable, the architecture is explicitly declaring that variable irrelevant to continuity. If I keeps a variable, changing it becomes a detectable continuity fault. Candidate invariants include lineage/event ancestry, semantic commitments, hashes, parent relationships, ordered checkpoints, and selected state-geometry summaries.

4.1 Direct versus indirect reconstruction

For three strata, two routes to C can be compared:

```
x_C^(direct) = g_AC(x_A)
x_C^(indirect) = g_BC(g_AB(x_A))
Delta_C = d_Z( I_C(x_C^(direct)), I_C(x_C^(indirect)) )
```

This is the head-tail test in a path language: independent reconstruction routes should meet on the same invariant content.

4.2 Closed-loop residual

```
H_ABC = g_CA o g_BC o g_AB
Omega_ABC(x) = d_Z( I_A(H_ABC(x)), I_A(H_ABC^0(x)) )
```

H_ABC^0 is the expected loop transform. For a purely representational loop it may be the identity. For a loop containing legitimate evolution, rotation, or phase accumulation, H_ABC^0 may be nontrivial. This prevents valid history from being mislabeled as corruption.

This yields one compact statement of the emerging principle:

> **Continuity is not raw-state identity. Continuity is path-consistent recoverability of chosen invariants.**

5. Four-stratum tetrahedron as a fault-localization code

With four strata A, B, C, D there are six pairwise transition edges and four triangular loops. Under the simple sparse-fault assumption that one transition is faulty and that the fault changes the invariant space, the pattern of failed loops identifies the edge.

Faulty edge	ABC	ABD	ACD	BCD	3-loop syndrome
AB	1	1	0	0	110
AC	1	0	1	0	101
AD	0	1	1	0	011
BC	1	0	0	1	100
BD	0	1	0	1	010
CD	0	0	1	1	001

Only three loops (ABC, ABD, ACD) are required to give six unique single-edge syndromes. The fourth loop BCD is then available as an additional consistency/parity check. For multiple simultaneous faults, binary syndromes can become ambiguous; retain the real-valued residual vector rather than thresholding too early.

```
r = [Omega_ABC, Omega_ABD, Omega_ACD, Omega_BCD]^T
Near a healthy state: r ~= M e
```

Here e is a six-edge fault vector and M is the edge-to-loop incidence/sensitivity matrix. This opens the door to network tomography and sparse-error localization rather than mere pass/fail checking.

6. Discrete cochain and gauge formulations

6.1 Linearized abelian case

Choose oriented edges [AB, AC, AD, BC, BD, CD] and oriented triangular faces [ABC, ABD, ACD, BCD]. Let e be an additive edge-error 1-cochain. The face curvature f is its discrete coboundary:

```
f = B e
```

	AB	AC	AD	BC	BD	CD
ABC	+1	-1	0	+1	0	0
ABD	+1	0	-1	0	+1	0
ACD	0	+1	-1	0	0	+1
BCD	0	0	0	+1	-1	+1

Because the oriented boundary of a boundary is zero, these four face residuals obey one parity relation:

```
-f_ABC + f_ABD - f_ACD + f_BCD = 0
```

This algebraic dependency is the formal reason the fourth face can act as a consistency check in the ideal linearized model.

6.2 Non-abelian case

When transitions are rotations or other noncommuting transforms, additive edge errors are insufficient. Assign each oriented edge a group-valued transporter U_ij. For a triangle:

```
H_ABC = U_CA U_BC U_AB
```

Under a local change of representation h_i at each stratum,

```
U_ij -> h_j U_ij h_i^(-1)
H_ABC -> h_A H_ABC h_A^(-1)
```

Individual edge representations are frame-dependent, while conjugacy-invariant properties of loop holonomy can be frame-independent. The simple additive parity equation no longer survives literally. Its structural replacement is a discrete Bianchi-type consistency relation: appropriately oriented face holonomies, parallel-transported to a common basepoint, compose to the expected identity/curvature constraint around the tetrahedral boundary.

> **Interpretation:** groupoid describes the strata and admissible arrows; cocycle describes compatibility; gauge connection describes transported state; holonomy is the loop defect; the invariant extractor determines which defects count as continuity failures.

7. Minimal continuity architecture

The live observation suggests a practical architecture for local/offline Hearth-style continuity. The root should not be any one model context, ANN index, summary, device, or UI. Those are projections. The root should be a provenance-preserved event lineage from which useful projections can be rebuilt.

Layer	Role
Append-only event spine / DAG	Primary lineage and causal ancestry; immutable event IDs; content hashes.
Logical chronology	Lamport index and/or version vectors for causal ordering and concurrent-branch detection.
Snapshots / anchors	Compressed checkpoints for fast re-entry; never the sole source of truth.
ANN / semantic index	Searchable projection over source events; rebuildable when embedding models change.
Active model context	Temporary working projection for one HearthKin/client.
Invariant extractor I	Defines continuity-critical content and ignores harmless frame/UI differences.
Loop monitor	Compares independent reconstruction paths and records residual syndromes.

7.1 Minimal experiment for the current system

- Confirm that app and browser links resolve to the same conversation identifier when possible.
- Choose three or four clients/surfaces and label them A, B, C, D.
- Create a uniquely tagged event on one client; record local observation time, visible tip event, and client metadata.
- Observe propagation to the other strata without manually refreshing at first; record version lag and rendering differences separately.
- Create a controlled stale-view or offline interval if the interface permits it, then resynchronize.
- For each route, reconstruct the chosen invariant state and compute direct/indirect residuals.
- Repeat after closing/reopening clients to test rehydration/consolidation.
- Only after repeated trials estimate thresholds epsilon and decide which residuals are normal synchronization noise.

A later local implementation can make this much cleaner because event IDs, version vectors, snapshot hashes, and transitions can all be logged directly rather than inferred from screenshots.

8. Falsification criteria and conclusions

The framework becomes useful only if it can fail. The following outcomes would count against a proposed continuity mapping rather than being explained away:

- Two views claimed to be the same lineage cannot be reconciled to compatible event ancestry.
- Independent reconstruction routes repeatedly produce invariant disagreement above a calibrated tolerance.
- A supposed frame-only transform changes variables that the architecture itself declares continuity-critical.
- Loop residuals remain inconsistent with any sparse or structured transition-fault model.
- Restart/replay from preserved anchors and event history cannot reproduce the claimed invariant state.
- Changing the invariant extractor after seeing failures is used to hide errors instead of being justified independently.

8.1 Working definitions

Temporal stratum	A provenance-tagged client-local observation of one evolving lineage at a particular synchronized version.
Consolidation	Visible convergence of previously divergent replicas after synchronization/reconciliation.
Continuity invariant	A property deliberately required to survive admissible changes of client, frame, path, or representation.
Loop residual	Distance between actual and expected invariant state after transport around a closed provenance path.
Continuity fault	A transition/path defect that changes continuity-critical invariant content beyond tolerance.

The central result of this note is compact:

> **Identity, current state, observation, and presentation are different layers. A continuity system should preserve recoverable lineage and invariants across those layers, then test itself by asking whether independent paths close.**

The live app/browser behavior supplied a small but concrete systems example: temporary divergence can coexist with a singular root, and later reconciliation can make that singularity visible again. The next step is not to assume more than the observation supports, but to instrument the same phenomenon with explicit event IDs, logical chronology, provenance, and measurable loop residuals.

End of working note v0.1